

PokeDex — Architecture

A vanilla-JS multi-page Pokédex. No build step, no framework, no backend. Four static HTML pages served as-is, each booting its own script. Data comes from the public PokéAPI; the only persisted state is a list of favorite IDs in `localStorage`.

Stack

- HTML5 (semantic structure, no templating)
- CSS3 (custom properties + Flexbox/Grid + media queries)
- Vanilla JavaScript (ES6+, `fetch`, `async/await`)
- PokéAPI — <https://pokeapi.co/api/v2/pokemon> (free, no key)
- `localStorage` for favorites persistence

File map

File	Role
<code>index.html</code>	Home grid — entry point
<code>pokemon.html</code>	Detail page for a single Pokémon
<code>favorites.html</code>	Grid of saved Pokémon
<code>about.html</code>	Static info page (no JS)
<code>styles.css</code>	Shared design system (loaded by every page)
<code>pokemon.css</code>	Detail-page-only styles (hero card, stat bars, cry button)
<code>script.js</code>	Logic for <code>index.html</code>
<code>pokemon.js</code>	Logic for <code>pokemon.html</code>
<code>favorites.js</code>	Logic for <code>favorites.html</code>

Architectural pattern

Each HTML page is an **isolated mini-app**. There is no module system, no shared JS bundle — every page boots its own script from scratch. Pages “communicate” only through:

1. **Navigation** — clicking a card uses `window.location.href = "pokemon.html?name=..."` rather than client-side routing.
2. **`localStorage["pokemon-favorites"]`** — a JSON array of Pokémon IDs, read/written by all three scripts.

This is why `getFavorites()` and `saveFavorites()` are duplicated across `script.js`, `pokemon.js`, and `favorites.js`.

Page-by-page summary

`index.html` + `script.js` — Home

Loads all 1025 Pokémon names up front, then fetches details in **batches of 40** as the user scrolls. Three flags drive the state machine:

- `isLoading` — guards against duplicate fetches
- `currentOffset` — pagination cursor
- `filteredMode` — when true, search/filter is active and infinite scroll is paused

Search and type filter only operate on **already-loaded** Pokémon — they don't refetch.

pokemon.html + pokemon.css + pokemon.js — Detail

Reads `?name=` from the URL (defaults to charizard). Single API call to `/pokemon/{name}`. Render highlights:

- Gradient header built from primary + secondary type colors
- Sprite swap on hover: official artwork ↔ animated Showdown sprite
- Stat bars animated via transition: width and a deferred `setTimeout`
- Cry sound playback if `pokemon.cries.latest` exists

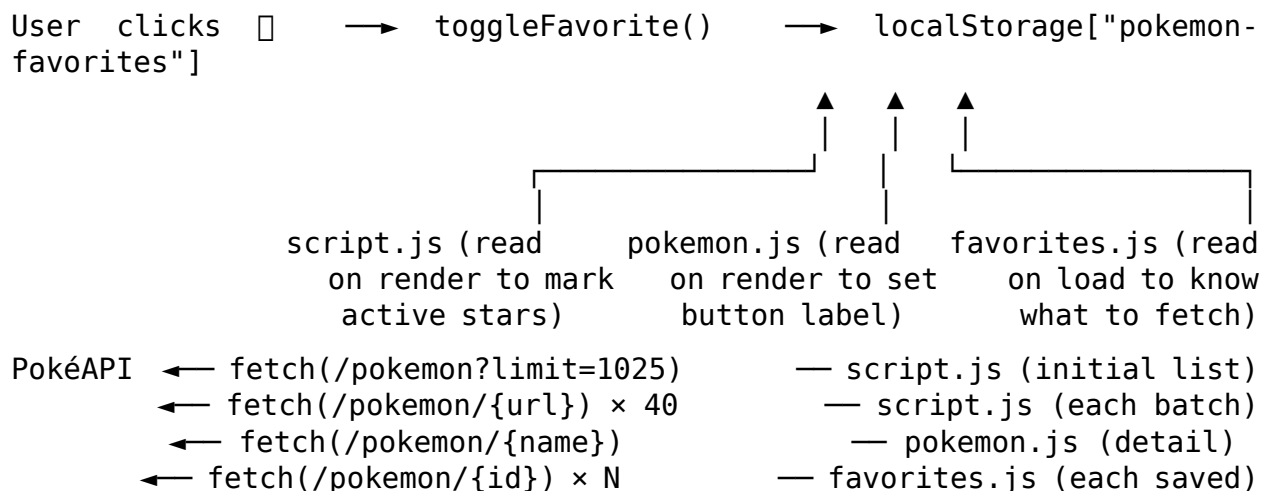
favorites.html + favorites.js — Saved

Reads IDs from `localStorage`, fetches each in parallel via `Promise.all`, renders the same card markup as the home grid. Removing a favorite mutates the DOM directly (no re-render).

about.html — Static

No JS. Three flexbox cards describing the stack, API, and features.

Data flow



Shared design system (styles.css)

CSS variables at `:root` define the theme:

```
--bg --surface --card --accent --accent2
--text --text-muted --border --shadow --radius
```

Layout is display: grid with repeat(5, 1fr) on desktop, dropping to 3 columns at $\leq 900\text{px}$ and 2 columns at $\leq 600\text{px}$. Type badges use hardcoded per-type background colors (.type-fire, .type-water, ...).

Note: the type color palette exists in two places — as CSS classes in styles.css (used by grid cards) and as the typeColors object in pokemon.js (used to compute the gradient header on the detail page). Hex values differ slightly between the two.

Known quirks

- **Search blind spot** — filterPokemon() on the home page only searches loaded Pokémon. Searching for #500 before scrolling that far returns nothing.
- **Duplicate favorites helpers** — getFavorites/saveFavorites redeclared in three files. Candidate for extraction.
- **Stale about copy** — about.html says “150 Pokémon loaded” but script.js has TOTAL = 1025.
- **Orphan modal** — index.html and favorites.html include <div id="modal"> markup, but no code path opens it. Dead leftover.
- **Sprite hover listener leak** — pokemon.js adds mouseenter/mouseleave listeners inside renderPokemon(). Searching a second Pokémon on the same page leaves the old listeners attached.
- **Inconsistent nav** — the four pages don’t agree on which links to show. pokemon.html drops the “Search” link; about.html is missing it too.